Name: Alba Samsami

Repo: https://github.com/albaparsi/Final-Project-Database

Requirements Gathering:

System Description

This project is a database system for a small online electronics store. The system will manage customers, staff members, products, inventory, and purchases. Staff will manage products and inventory, while customers will browse products and make purchases.

Data Requirements

The system will use four tables:

- **Customer:** Customer ID, first name, last name, email, phone number, and account creation date.
- **Staff:** Staff ID, first name, last name, email, job title, and hire date.
- **Product:** Product ID, name, description, category, price, quantity in stock.
- **Purchase:** Purchase ID, customer ID, product ID, quantity, total amount, payment method description, and purchase date.

Use Cases

User	Use case
Staff	Add a new product
Staff	Update a product's inventory quantity
Staff	View low-stock products
Customer / Staff	View or search available products
Staff	Add a new customer
Customer / Staff	Record a product purchase
Customer / Staff	View a customer's purchase history

Business Rules

- Customer and staff email addresses must be unique.

- Product prices must be greater than zero.
- Inventory quantities cannot be negative.
- A purchase quantity must be at least one.
- A purchase can only be made when enough inventory is available.
- Inventory is reduced when a purchase is completed.
- Full credit-card numbers and CVV values will not be stored.

Entities

- CUSTOMER: people who purchase products
- STAFF: employees who process or oversee purchases
- PRODUCT: items available in the online store
- PURCHASE: a transaction in which a customer buys one product

Relationships

- One customer can make zero or many purchases; each purchase belongs to one customer.
- One product can appear in zero or many purchases; each purchase includes one product.
- One staff member can process zero or many purchases; each purchase is processed by one staff member.

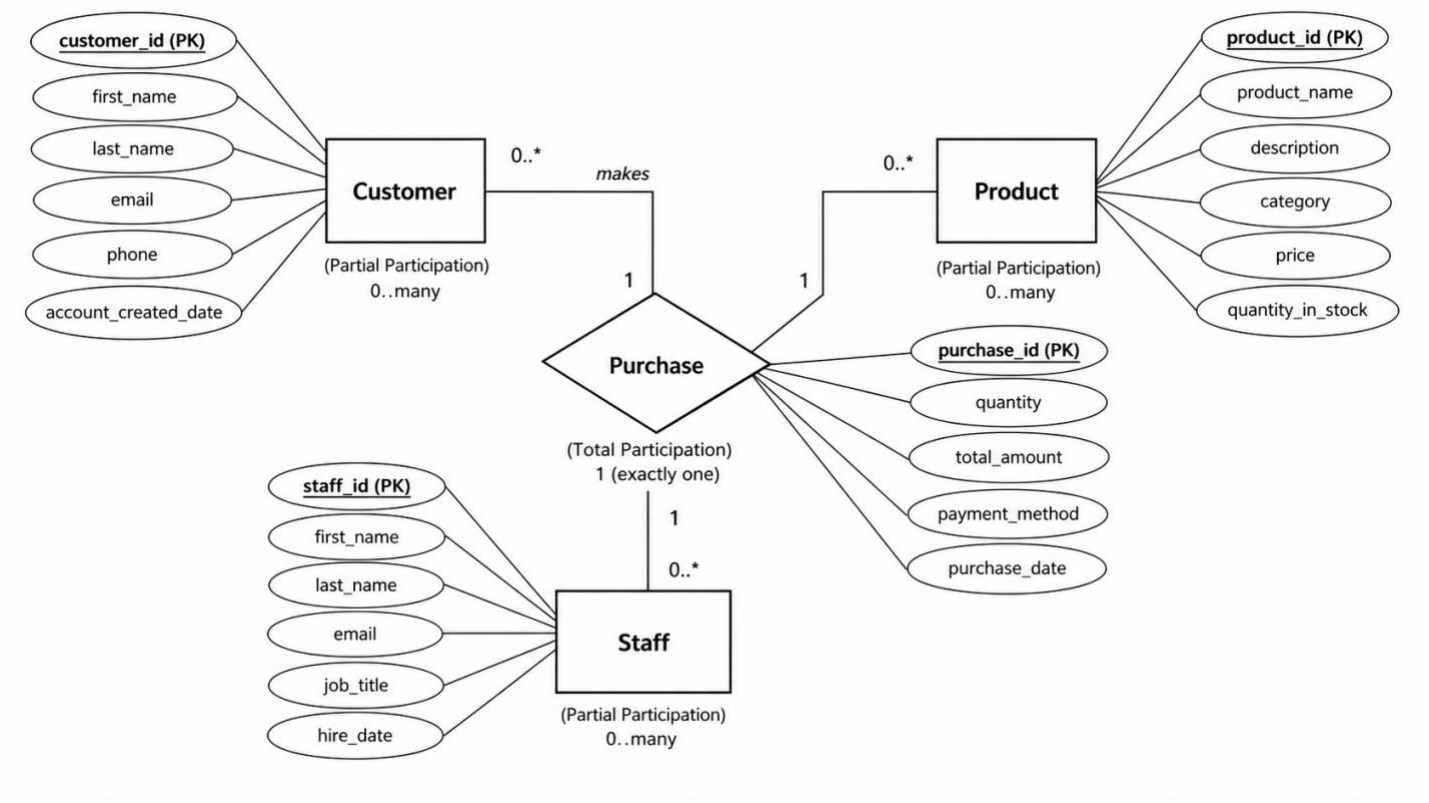
Keys

Entity	Primary key	Foreign keys
CUSTOMER	customer_id	None
STAFF	staff_id	None
PRODUCT	product_id	None
PURCHASE	purchase_id	customer_id, product_id, staff_id

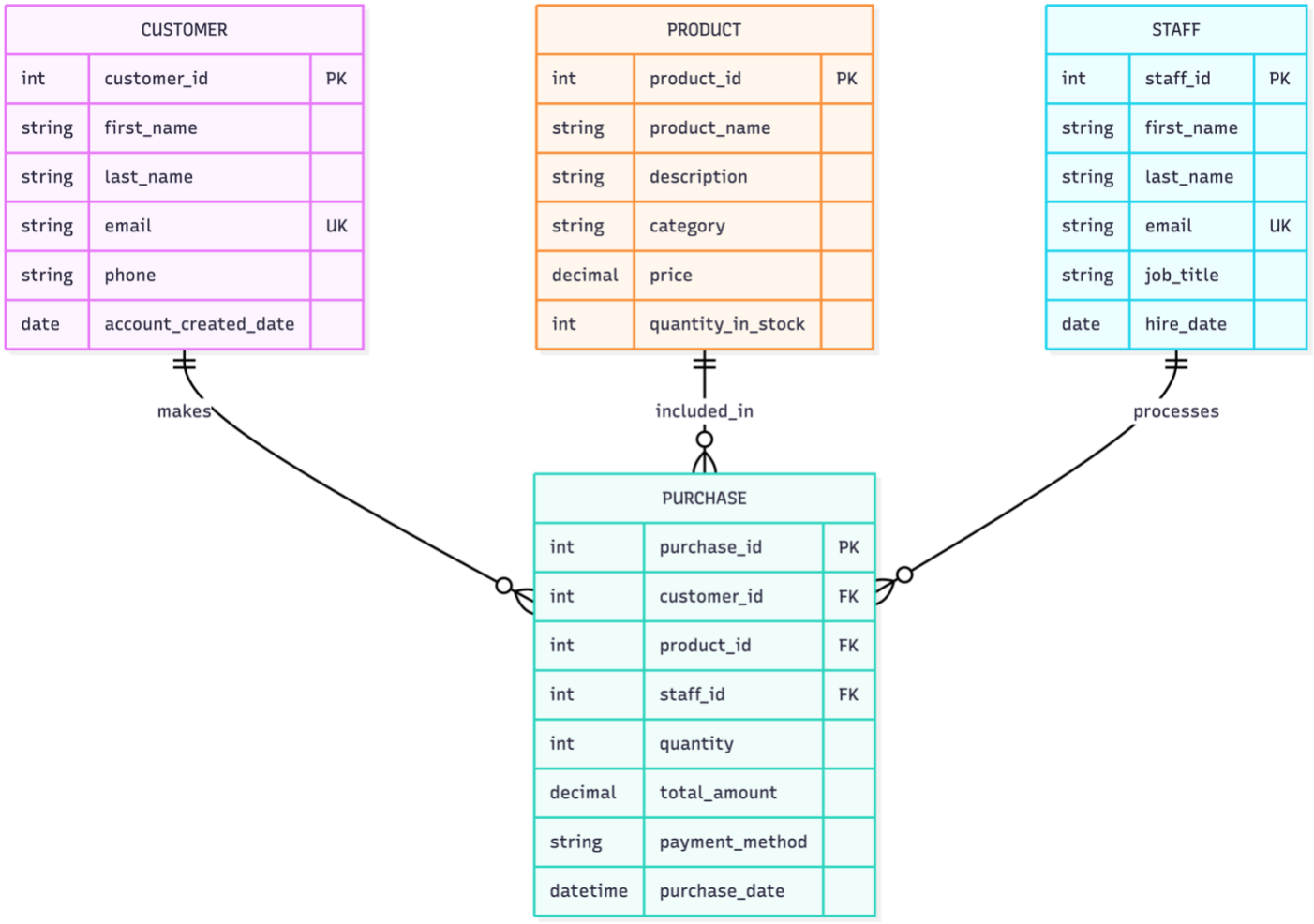
Cardinality and participation

Relationship	Cardinality	Participation
• Customer makes Purchase	• One-to-many: one customer can make 0 to many purchases; one purchase is made by exactly 1 customer	• Customer: partial, because a customer may never buy anything. Purchase: total, because every purchase must have a customer.
• Product included in Purchase	• One-to-many: one product can appear in 0 to many purchases; one purchase includes exactly 1 product	• Product: partial, because a product may never be purchased. Purchase: total, because every purchase must include a product.
• Staff processes Purchase	• One-to-many: one staff member can process 0 to many purchases; one purchase is processed by exactly 1 staff member	• Staff: partial, because a staff member may not have processed a purchase. Purchase: total, because each recorded purchase must be associated with a staff member.

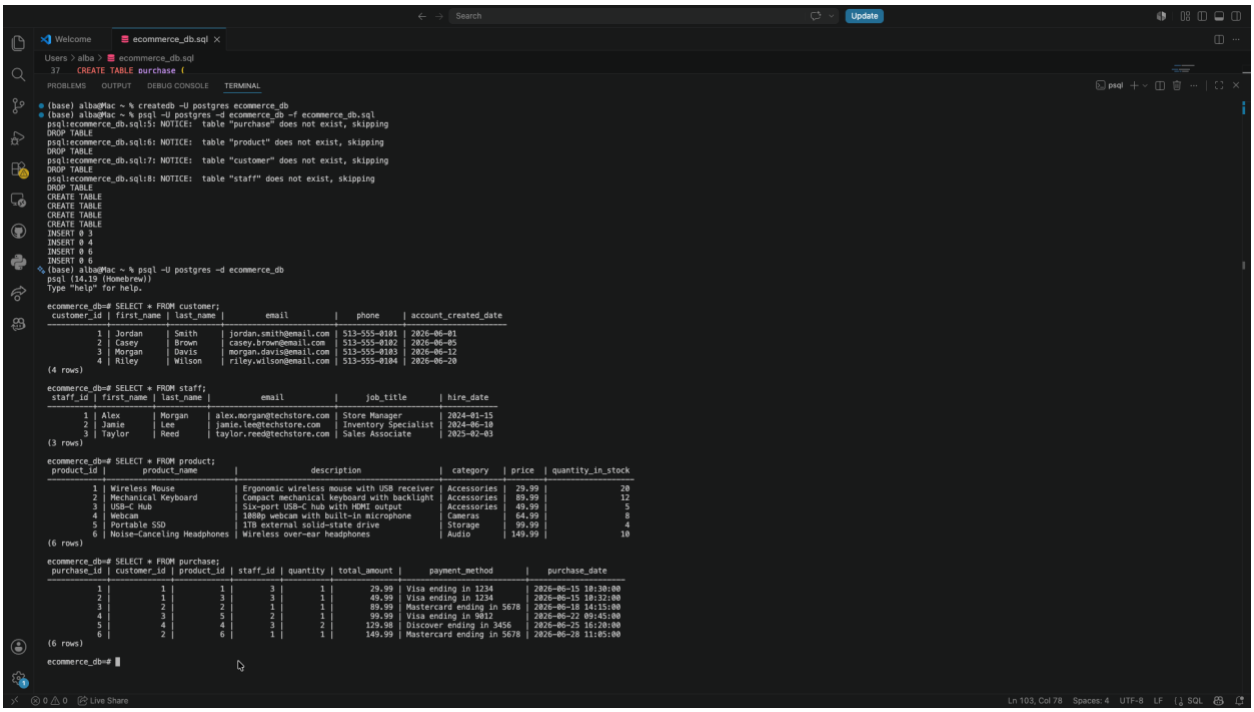
Entity-Relationship Diagram:



Schema Design:



Database Implementation:




```sql
-- E-Commerce Database System
-- CS4092 Final Project
-- PostgreSQL

DROP TABLE IF EXISTS purchase;
DROP TABLE IF EXISTS product;
DROP TABLE IF EXISTS customer;
DROP TABLE IF EXISTS staff;

CREATE TABLE staff (
    staff_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    job_title VARCHAR(75) NOT NULL,
    hire_date DATE NOT NULL
);
```

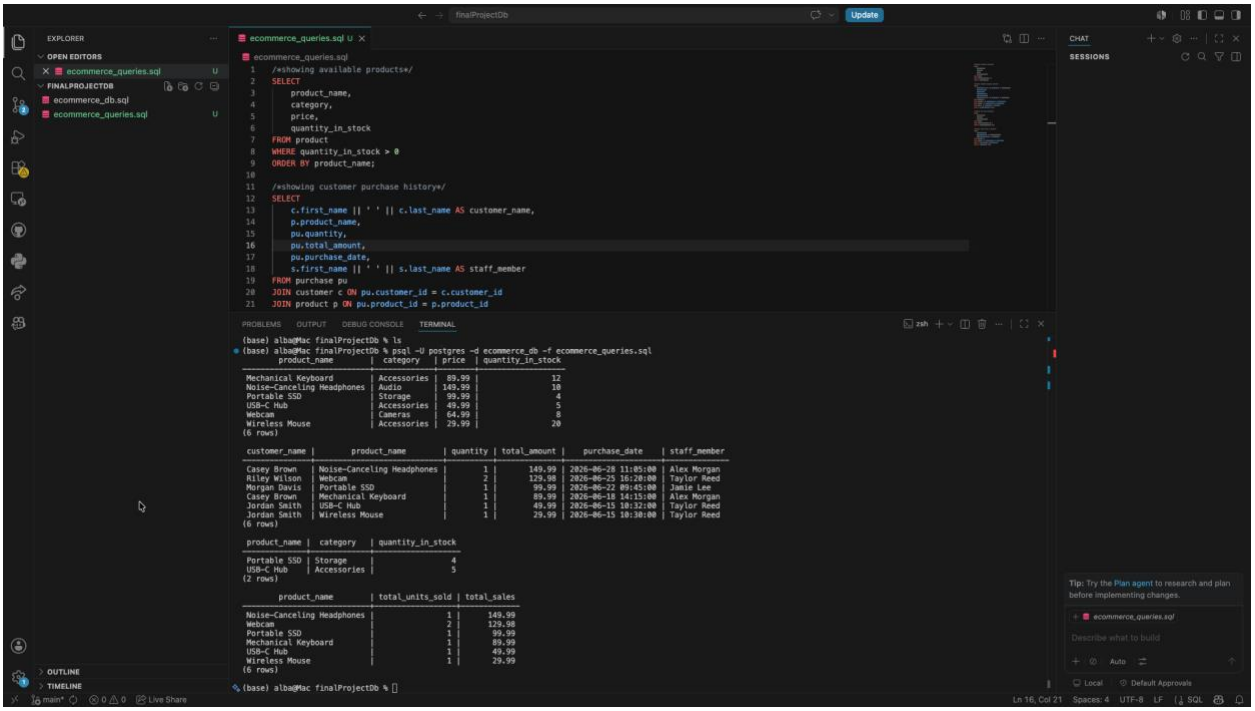
```sql
CREATE TABLE customer (
    customer_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    phone VARCHAR(20),
    account_created_date DATE NOT NULL DEFAULT CURRENT_DATE
);

CREATE TABLE product (
    product_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    description VARCHAR(255),
    category VARCHAR(50) NOT NULL,
    price DECIMAL(10,2) NOT NULL CHECK (price > 0),
    quantity_in_stock INT NOT NULL CHECK (quantity_in_stock >= 0)
);

CREATE TABLE purchase (
    purchase_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id INT NOT NULL,
    product_id INT NOT NULL,
    staff_id INT NOT NULL,
    quantity INT NOT NULL CHECK (quantity > 0),
    total_amount DECIMAL(10,2) NOT NULL CHECK (total_amount > 0),
    payment_method VARCHAR(50) NOT NULL,
    purchase_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_purchase_customer
        FOREIGN KEY (customer_id)
        REFERENCES customer(customer_id),

    CONSTRAINT fk_purchase_product
        FOREIGN KEY (product_id)
        REFERENCES product(product_id),

    CONSTRAINT fk_purchase_staff
```

```
    FOREIGN KEY (staff_id)
    REFERENCES staff(staff_id)
);

INSERT INTO staff (first_name, last_name, email, job_title, hire_date)
VALUES
  ('Alex', 'Morgan', 'alex.morgan@techstore.com', 'Store Manager', '2024-01-15'),
  ('Jamie', 'Lee', 'jamie.lee@techstore.com', 'Inventory Specialist', '2024-06-10'),
  ('Taylor', 'Reed', 'taylor.reed@techstore.com', 'Sales Associate', '2025-02-03');

INSERT INTO customer (first_name, last_name, email, phone, account_created_date)
VALUES
  ('Jordan', 'Smith', 'jordan.smith@email.com', '513-555-0101', '2026-06-01'),
  ('Casey', 'Brown', 'casey.brown@email.com', '513-555-0102', '2026-06-05'),
  ('Morgan', 'Davis', 'morgan.davis@email.com', '513-555-0103', '2026-06-12'),
  ('Riley', 'Wilson', 'riley.wilson@email.com', '513-555-0104', '2026-06-20');

INSERT INTO product (
  product_name,
  description,
  category,
  price,
  quantity_in_stock
)
VALUES
  ('Wireless Mouse', 'Ergonomic wireless mouse with USB receiver', 'Accessories', 29.99, 20),
  ('Mechanical Keyboard', 'Compact mechanical keyboard with backlight', 'Accessories', 89.99, 12),
  ('USB-C Hub', 'Six-port USB-C hub with HDMI output', 'Accessories', 49.99, 5),
  ('Webcam', '1080p webcam with built-in microphone', 'Cameras', 64.99, 8),
  ('Portable SSD', '1TB external solid-state drive', 'Storage', 99.99, 4),
  ('Noise-Canceling Headphones', 'Wireless over-ear headphones', 'Audio', 149.99, 10);

INSERT INTO purchase (
  customer_id,
  product_id,
  staff_id,
  quantity,
```

```
    total_amount,
    payment_method,
    purchase_date
)
VALUES
    (1, 1, 3, 1, 29.99, 'Visa ending in 1234', '2026-06-15 10:30:00'),
    (1, 3, 3, 1, 49.99, 'Visa ending in 1234', '2026-06-15 10:32:00'),
    (2, 2, 1, 1, 89.99, 'Mastercard ending in 5678', '2026-06-18 14:15:00'),
    (3, 5, 2, 1, 99.99, 'Visa ending in 9012', '2026-06-22 09:45:00'),
    (4, 4, 3, 2, 129.98, 'Discover ending in 3456', '2026-06-25 16:20:00'),
    (2, 6, 1, 1, 149.99, 'Mastercard ending in 5678', '2026-06-28 11:05:00');
```

Database Interaction:

SQL Queries:




```
/*showing available products*/
SELECT
    product_name,
    category,
    price,
```

```sql
    quantity_in_stock
FROM product
WHERE quantity_in_stock > 0
ORDER BY product_name;

/*showing customer purchase history*/
SELECT
    c.first_name || ' ' || c.last_name AS customer_name,
    p.product_name,
    pu.quantity,
    pu.total_amount,
    pu.purchase_date,
    s.first_name || ' ' || s.last_name AS staff_member
FROM purchase pu
JOIN customer c ON pu.customer_id = c.customer_id
JOIN product p ON pu.product_id = p.product_id
JOIN staff s ON pu.staff_id = s.staff_id
ORDER BY pu.purchase_date DESC;

/*showing low stock products*/
SELECT
    product_name,
    category,
    quantity_in_stock
FROM product
WHERE quantity_in_stock <= 5
ORDER BY quantity_in_stock ASC;

/*showing total sales by product*/
SELECT
    p.product_name,
    SUM(pu.quantity) AS total_units_sold,
    SUM(pu.total_amount) AS total_sales
FROM purchase pu
JOIN product p ON pu.product_id = p.product_id
GROUP BY p.product_id, p.product_name
ORDER BY total_sales DESC;
```

Business Logic:

Added to github repo.